

香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen



人工智能學院
School of Artificial Intelligence

AIE1901 -S1 Lifelong AI: Lecture 3: The Out-of-Distribution (OOD) Problem & Transfer Learning

Yang Li

2026/10/12

Agenda

- Supervised Learning Review
- What is OOD?
- A simple transfer learning algorithm: finetuning

Review: Supervised Learning

A **training dataset** $D_{\text{train}} = (X, Y)$ is a collection of input and labels:

$$X = \{x^{(1)}, \dots, x^{(m)}\},$$
$$Y = \{y^{(1)}, \dots, y^{(m)}\}$$

Goal: Learn the parameters of a hypothesis function h that can correct predict the label of any new "test" samples (i.e. $h(x^{(\text{test})}) = y^{(\text{test})}$)

i.i.d. assumption (independent and identically distributed)

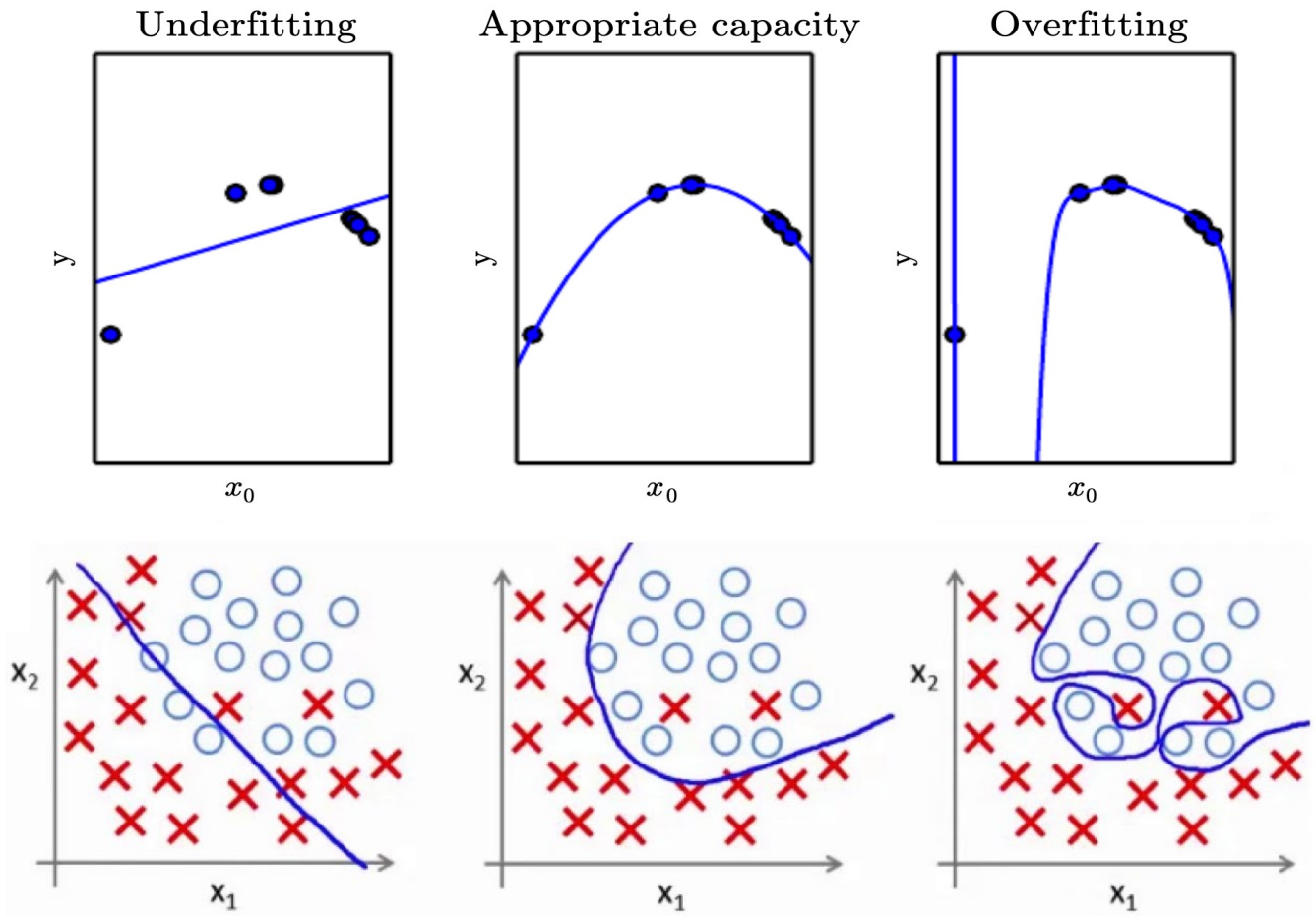
- **Independent:** Each data point does not influence other data points
- **Identically distributed**: Each sample is drawn from the same underlying probability distribution $P(X, Y)$



in-distribution learning

Review: Overfit & Underfit

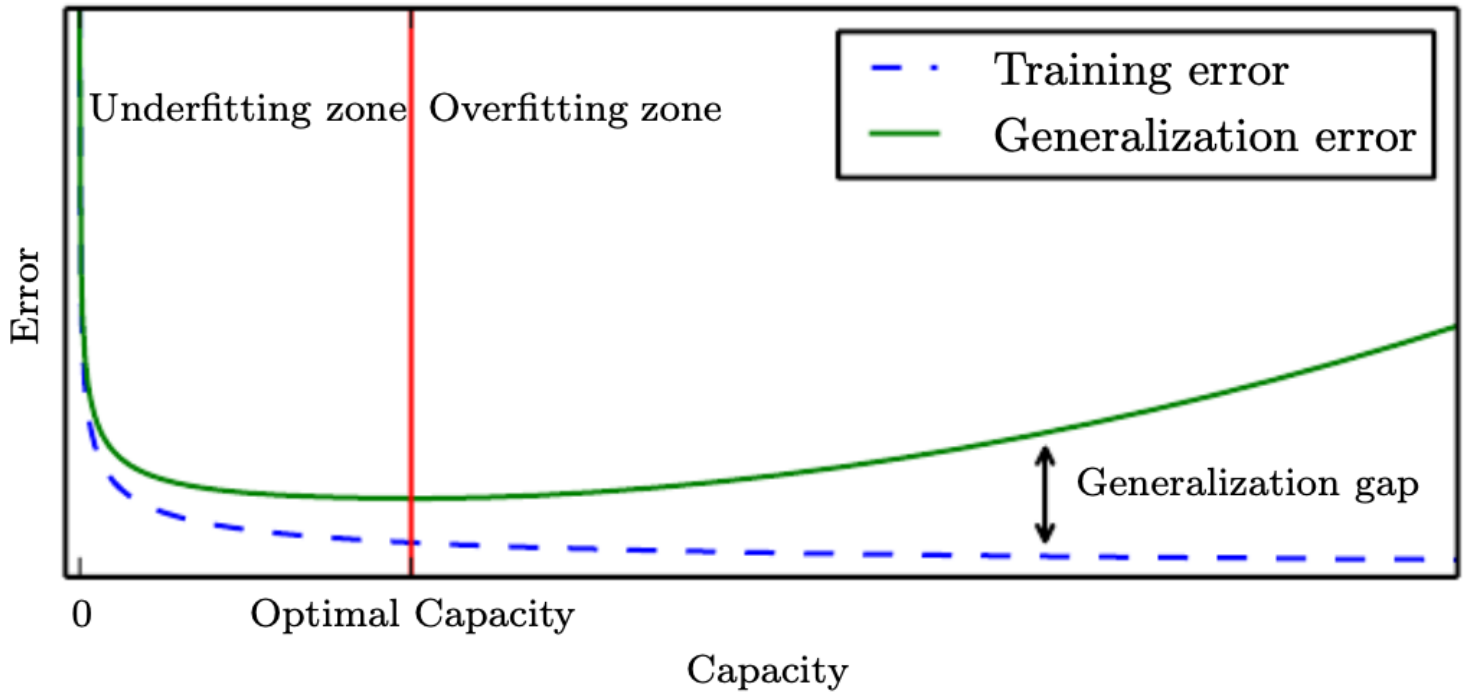
- **Underfitting**: Both training error and testing error are large
- **Overfitting:** Training error is small, testing error is large



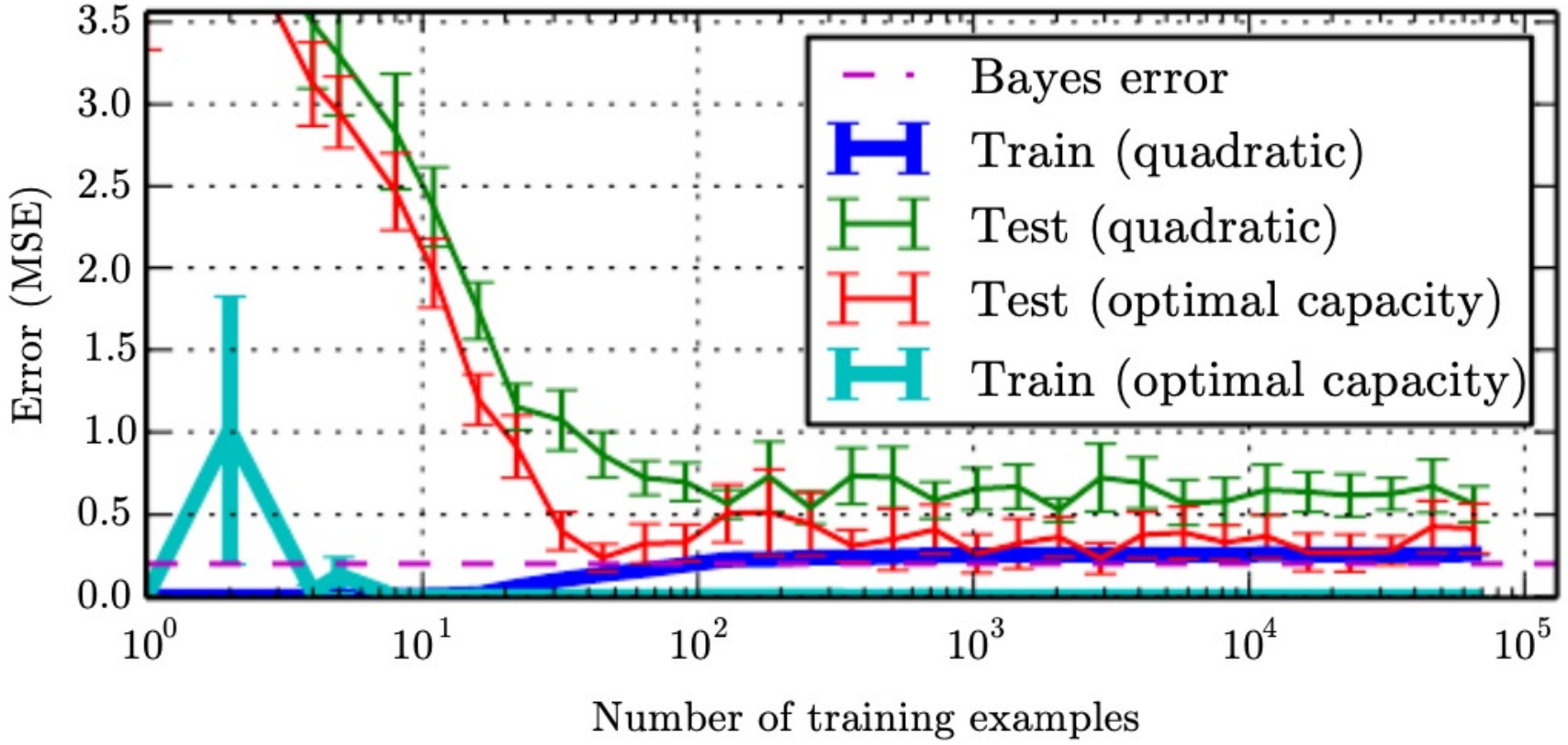
Review: Impact of Model Capacity

As model capacity (complexity) increases

- **Training error** decreases — harder to underfit
- The **generalization gap** (between training and testing error) increases — easier to overfit



Review: Impact of Sample Size



- **More data improves generalization:** More data reduces test error and the train–test gap.
- **More data supports more complex models:** Optimal model capacity increases with dataset size, then plateaus once sufficient capacity is reached.

Out-of-Distribution (OOD) Problem

$$P_{\text{test}}(X, Y) \neq P_{\text{train}}(X, Y)$$

Ways to handle OOD: **Data-centric approaches**

- Increase training data diversity (e.g. make $P_{\text{train}}(X, Y) \approx P_{\text{test}}(X, Y)$) Takes longer to train
- Train a new model from scratch using newly collected training samples

Slow and requires large samples size

Types of Distribution Shifts

$$P_{\text{test}}(X, Y) \neq P_{\text{train}}(X, Y)$$



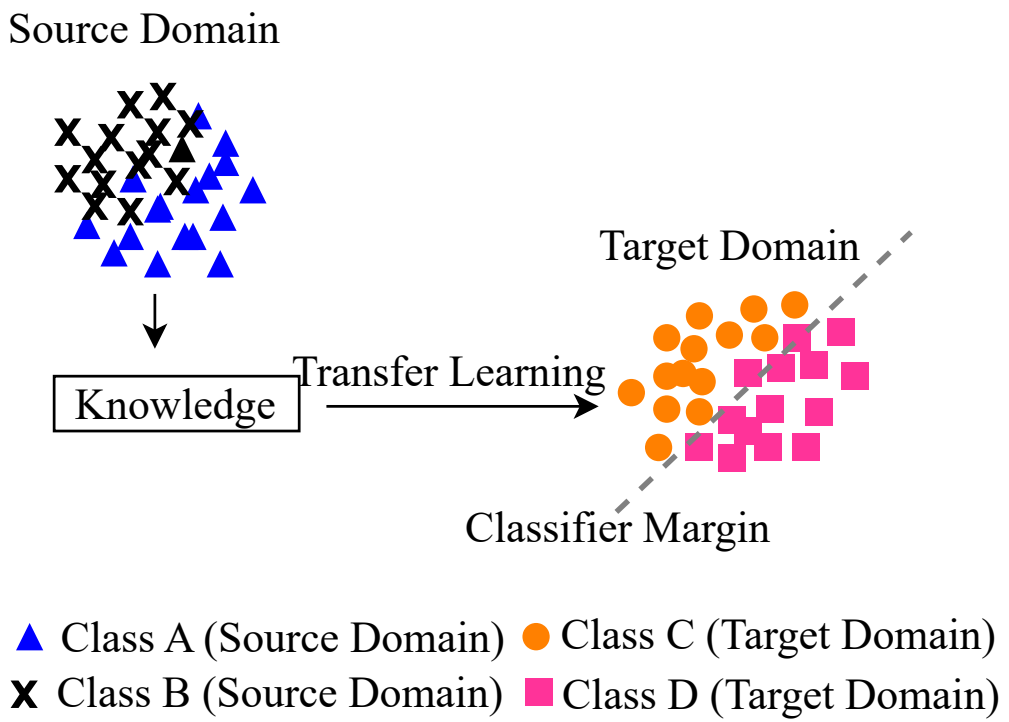
Covariate (Feature) Shift: $P_{\text{test}}(X) \neq P_{\text{train}}(X), \mathcal{Y}_{\text{train}} = \mathcal{Y}_{\text{test}}, P_{\text{test}}(Y|X) = P_{\text{train}}(Y|X)$

Semantic Shift: $\mathcal{Y}_{\text{train}} \neq \mathcal{Y}_{\text{test}}$ or $P_{\text{test}}(Y|X) \neq P_{\text{train}}(Y|X), P_{\text{test}}(X) = P_{\text{train}}(X)$

Out-of-Distribution (OOD) Problem

Ways to handle OOD:

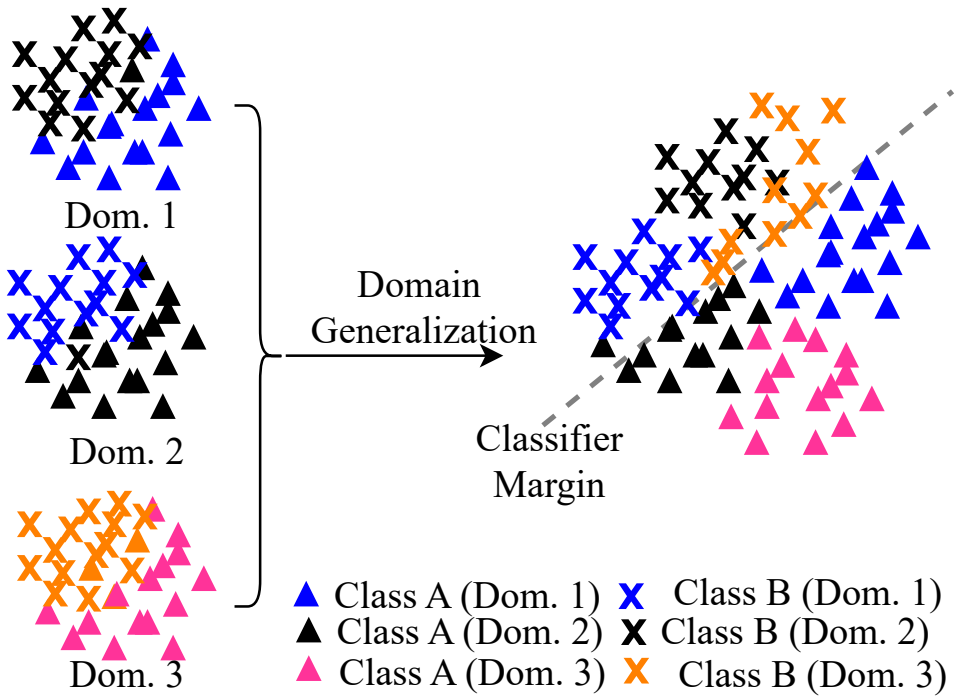
- **Transfer Learning**: Use knowledge from a **source task** to improve the learning of the **target task**



Out-of-Distribution (OOD) Problem

Ways to handle OOD:

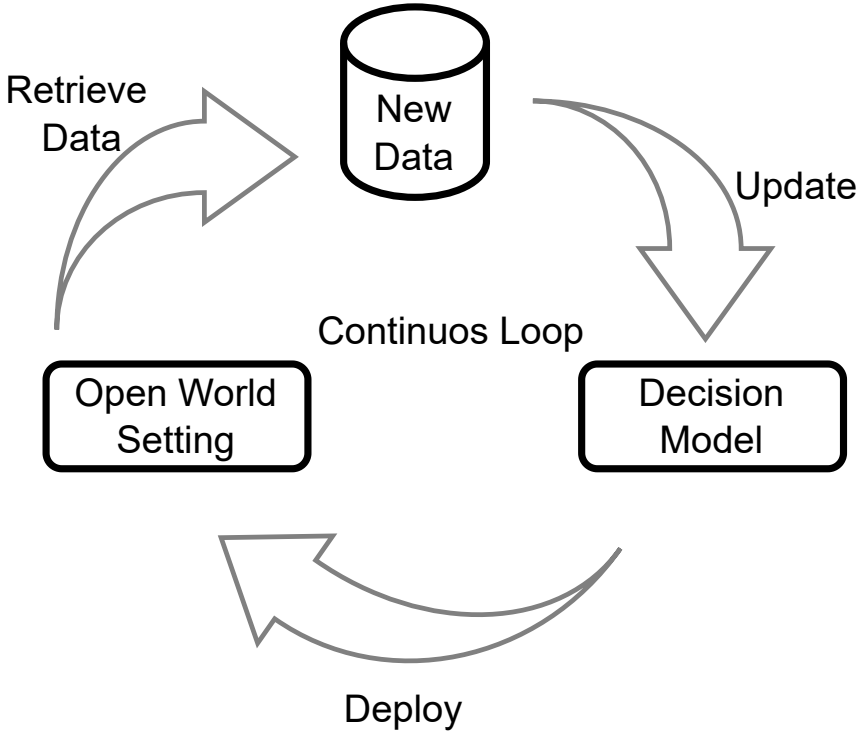
- **Transfer Learning**: Use knowledge from a **source task** to improve the learning of the **target task**
- **Domain Generalization**: Use data from one or more source domains to generalize to unseen target domains



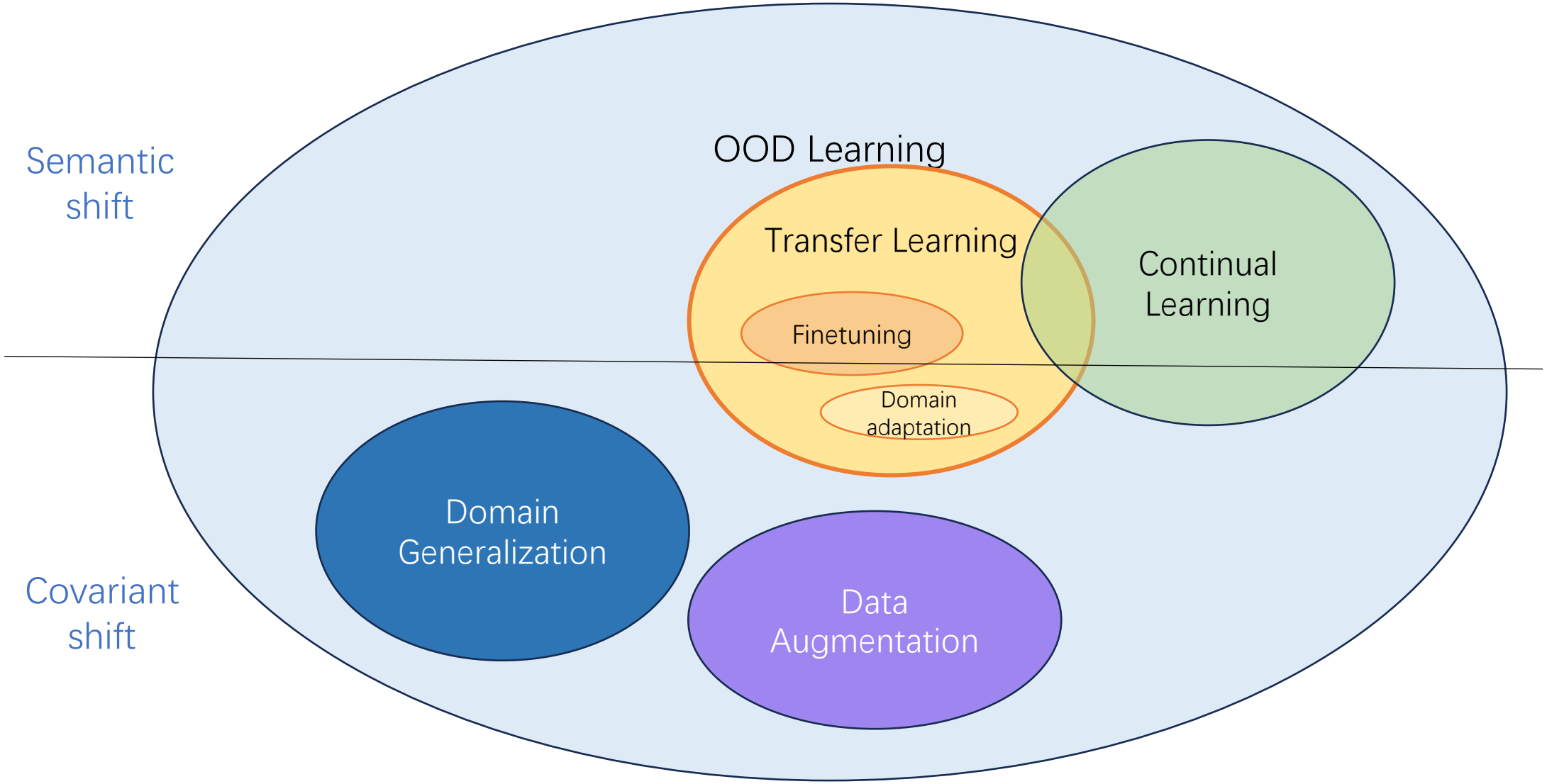
Out-of-Distribution (OOD) Problem

Ways to handle OOD:

- **Transfer Learning**: Use knowledge from a **source domain** to improve the learning of the **target domain**
- **Domain Generalization**: Use data from one or more source domains to generalize to **unseen target domains**
- **Continual Learning**: **Continuously update** model from changing environment to **accumulate knowledge over time**



Out-of-Distribution (OOD) Problem



Banana Ripeness Classification



Source dataset (BananaImageBD [Ferdaus 2024])

Green



Overripe



Ripe



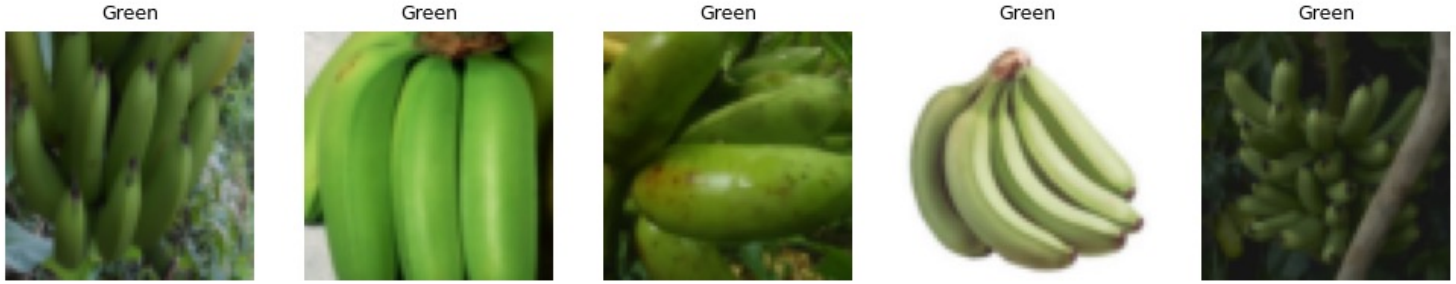
295 training samples

Banana Ripeness Classification



Target dataset (Banana Ripeness Classification by Carvalho)

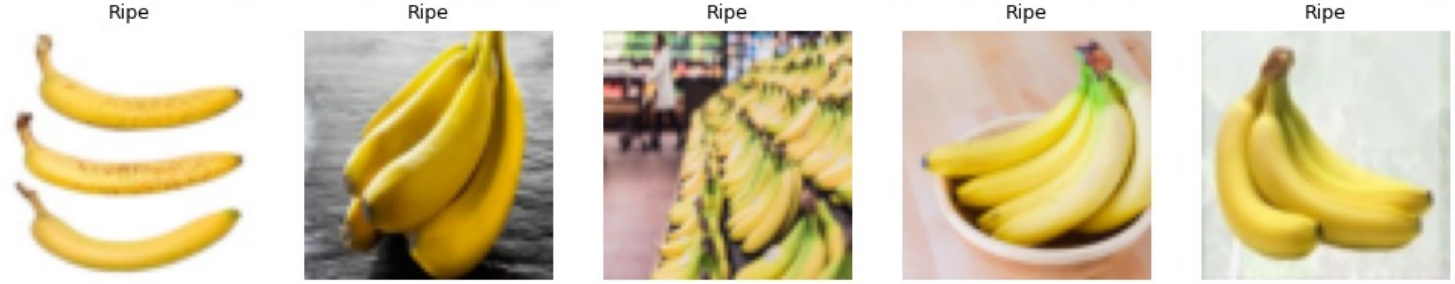
Green



Overripe



Ripe



152 images

- 76 Training
- 76 Testing

Simple Feed-Forward Neural Network

```python
class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.feature_extractor = nn.Sequential(
          nn.Flatten(),

          nn.Linear(3 * 64 * 64, 32),
          nn.ReLU(),

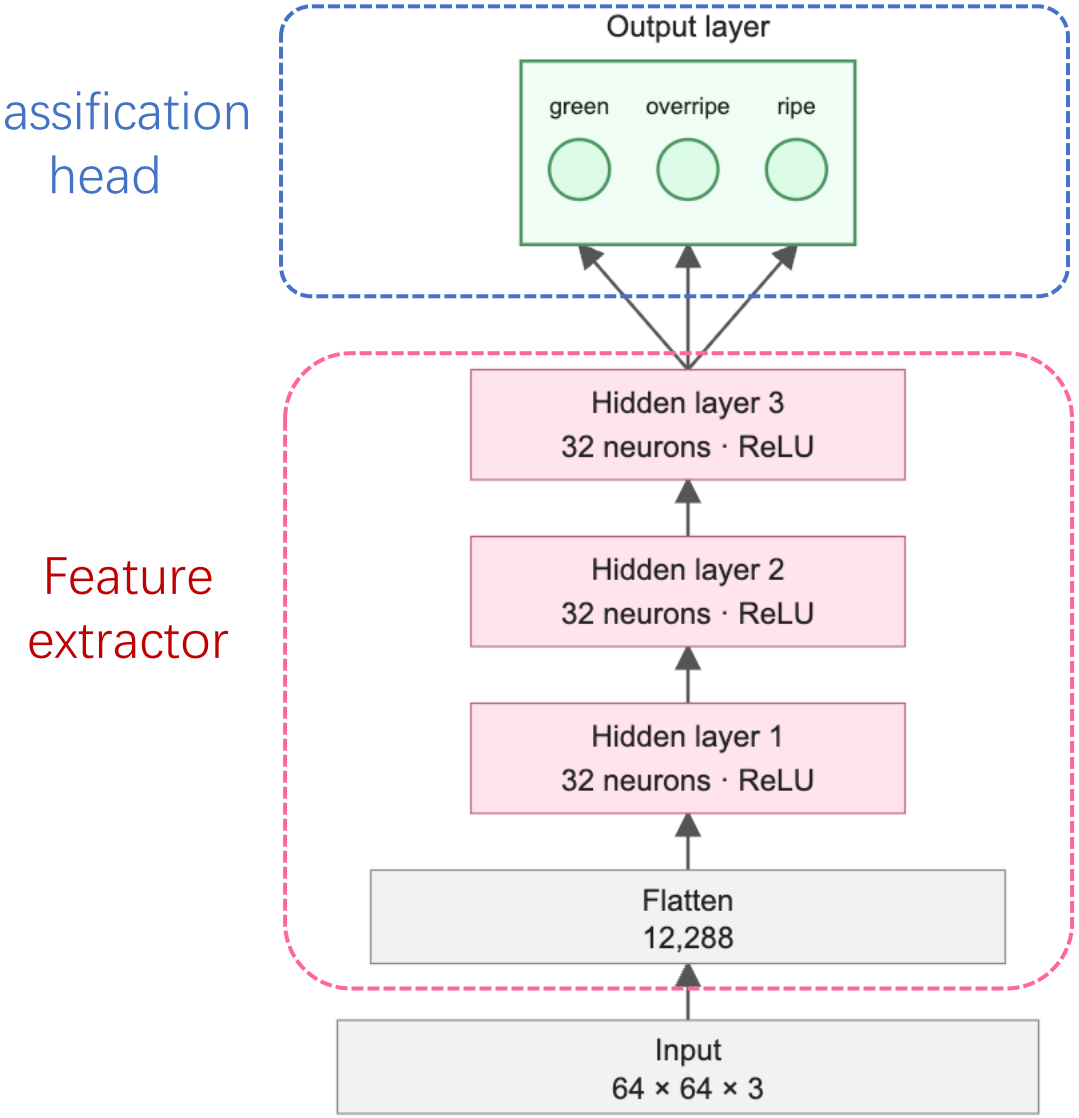
          nn.Linear(32, 32),
          nn.ReLU(),

          nn.Linear(32, 32),
          nn.ReLU()
        )

        # Classification head
        self.classifier = nn.Linear(32, 3)

    def forward(self, x):
        features = self.feature_extractor(x)
        logits = self.classifier(features)

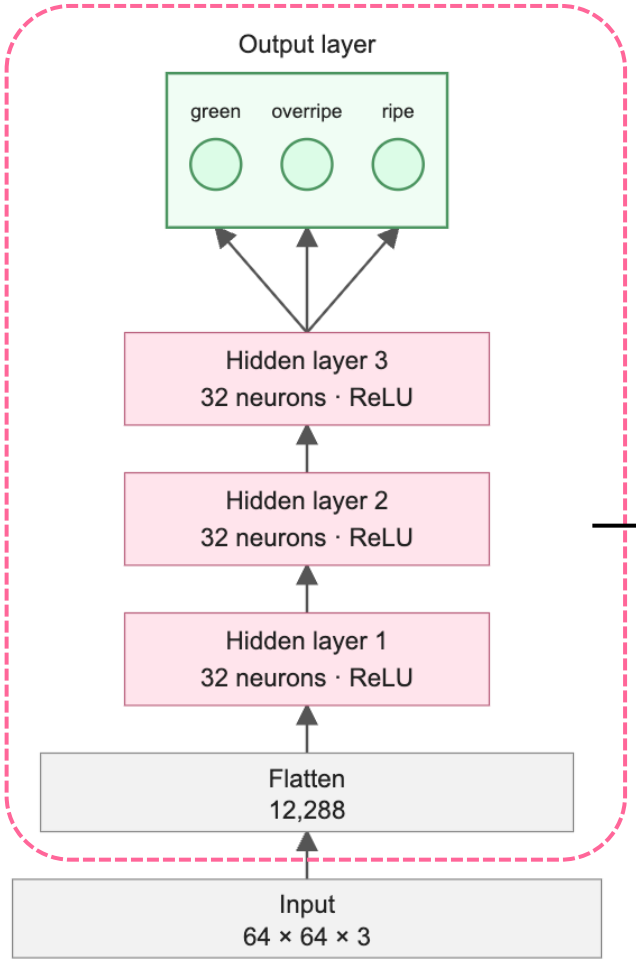
        return logits
```



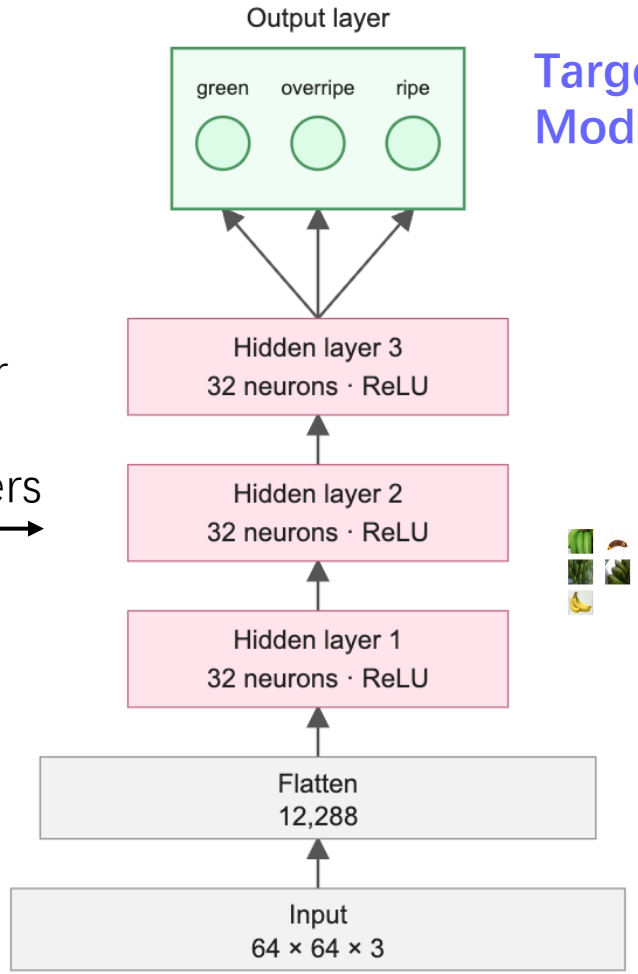
1st Transfer Learning Algorithm: Finetuning

Source Model





Transfer model parameters

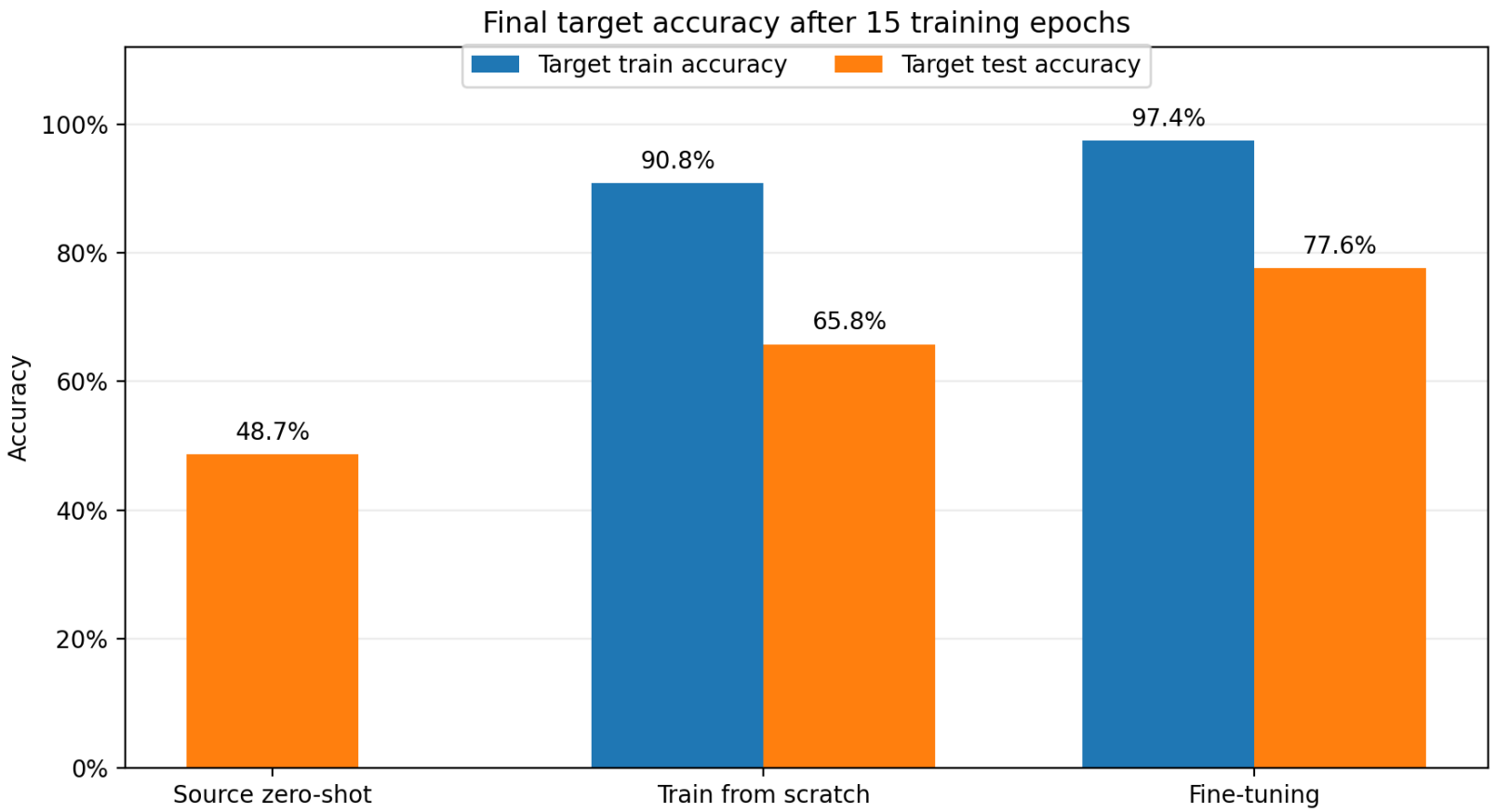


Target Model

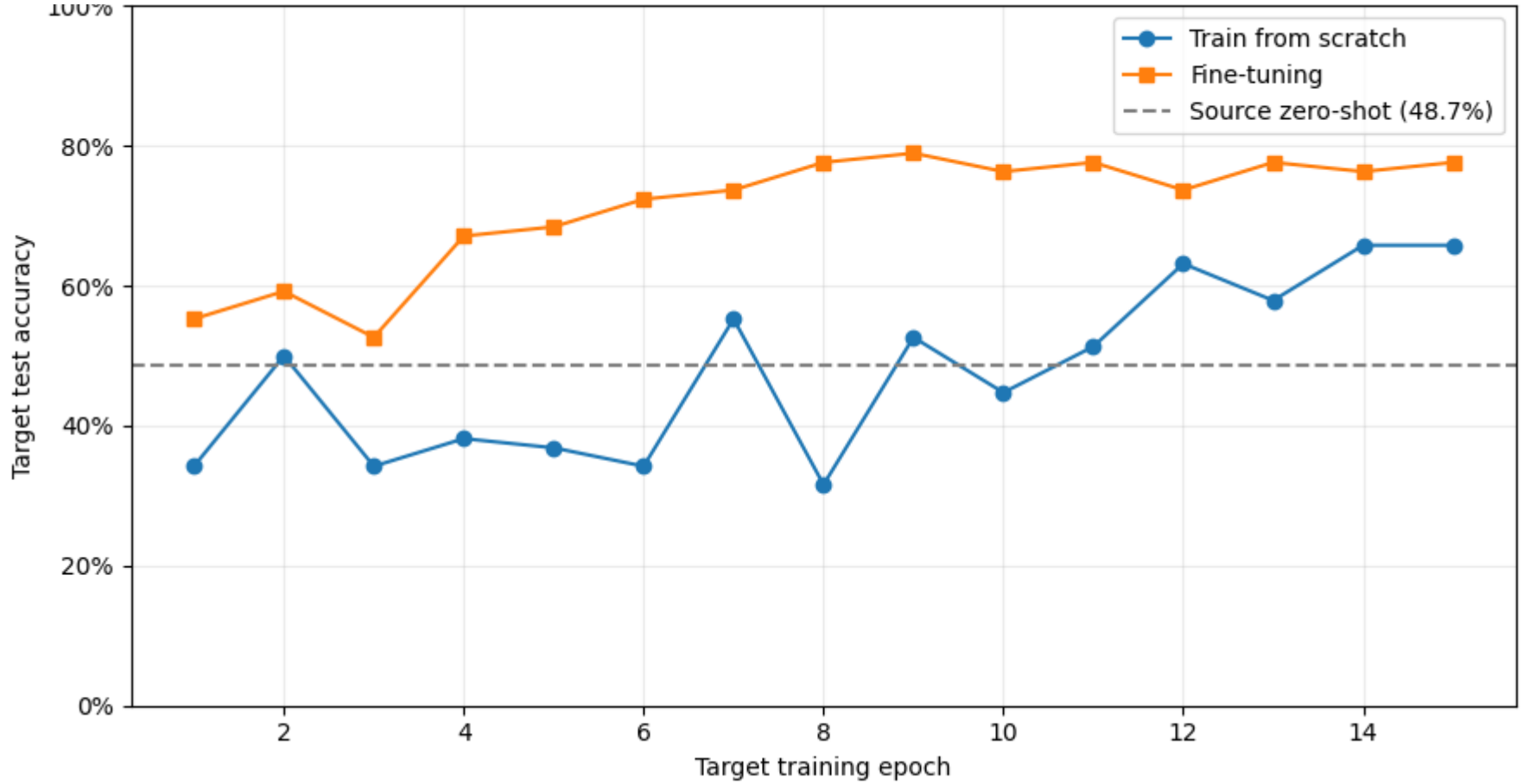


- **Step 1**: Initialize target model feature extractor with source model parameters
- **Step 2**: Continual update model parameters using target training data

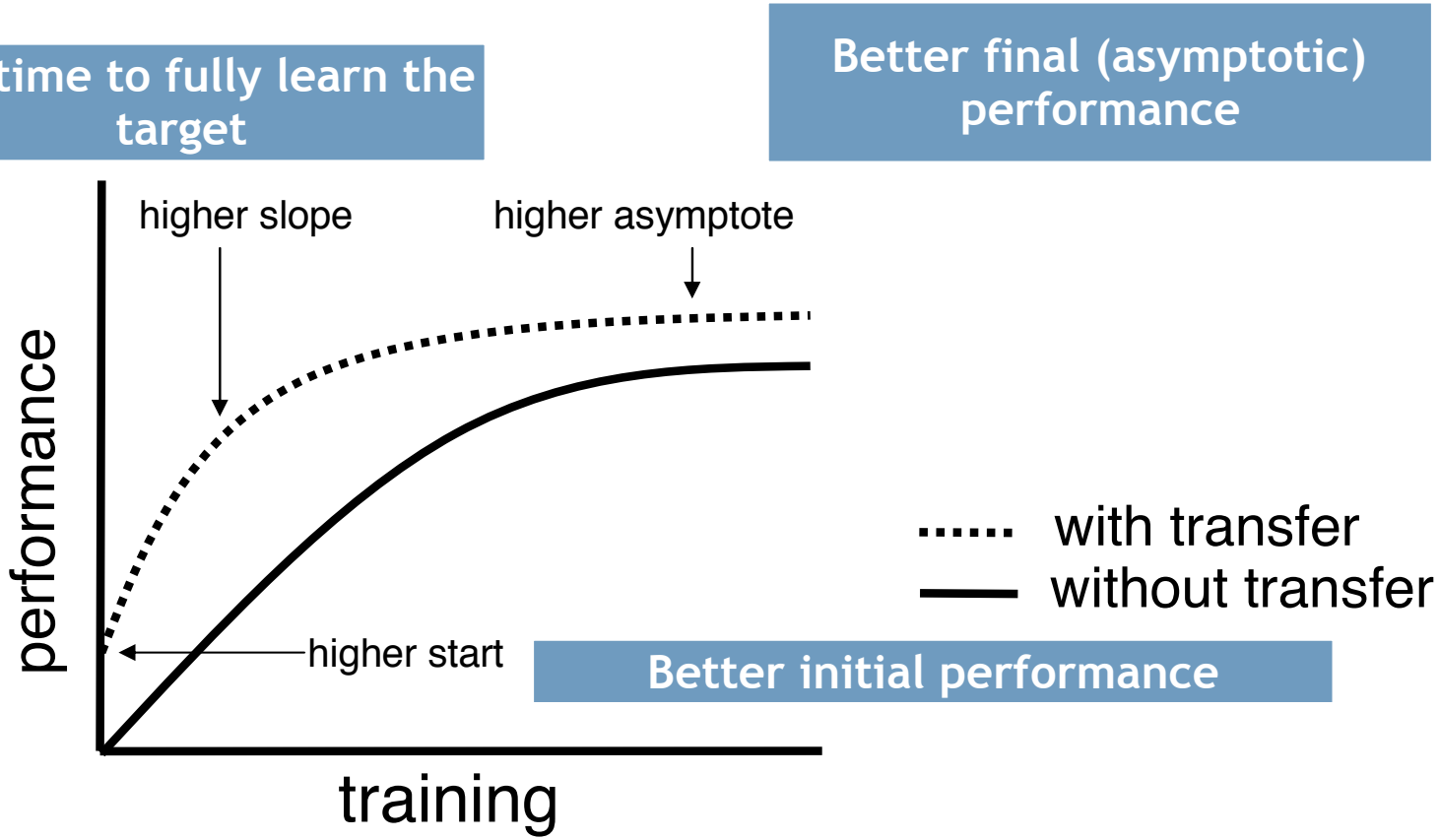
Finetuning vs. Training from Scratch



Finetuning vs. Training from Scratch

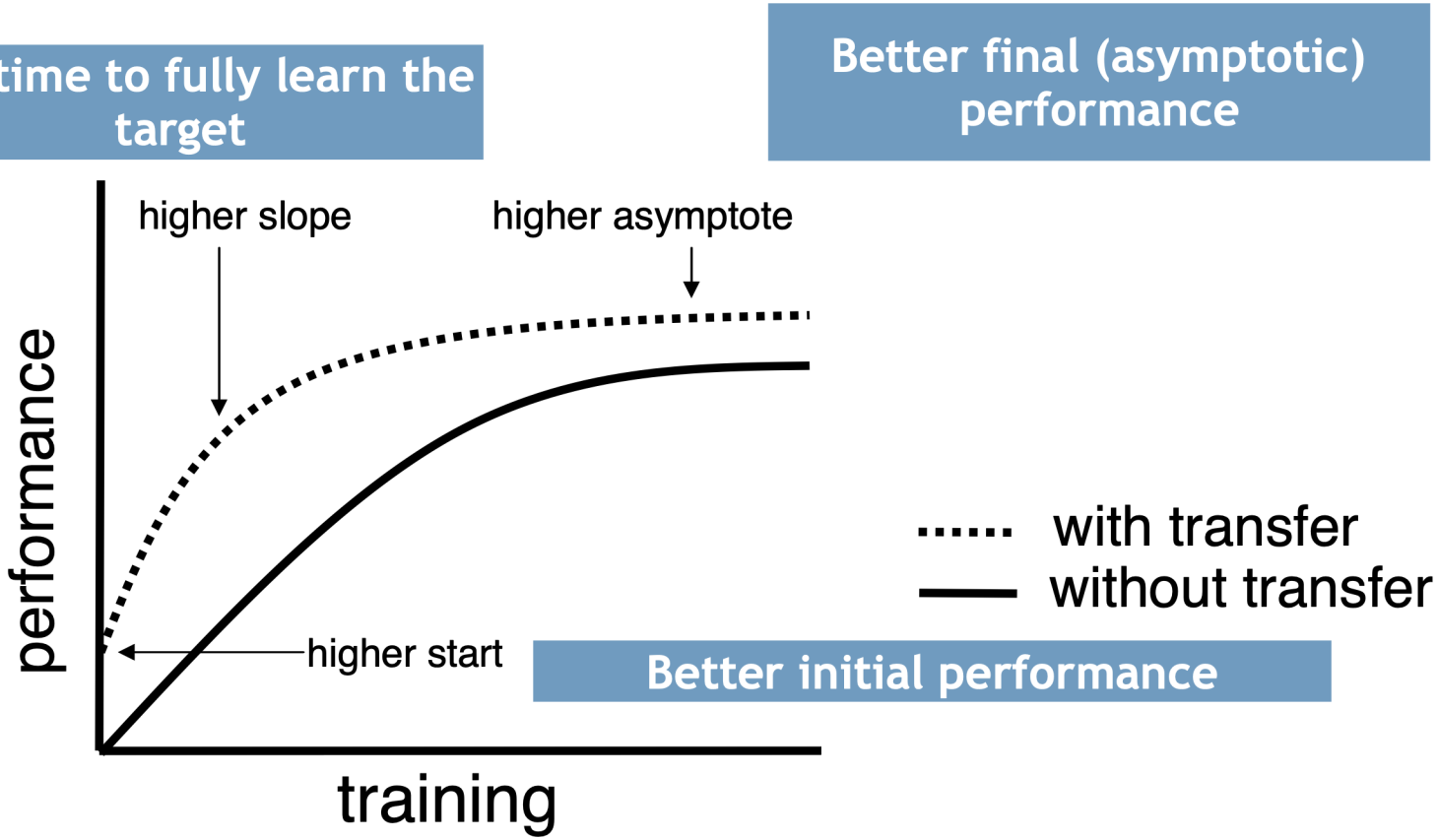


Summary



- **Compared to training from scratch,** transfer learning can provide
  - Higher initial performance
  - Faster learning
  - Better final performance

Summary



- **Questions**
  - How does sample size affect transfer effectiveness?
  - Should we always finetuning all parameters?
  - When would transfer learning fail?